

MMC-HVDC Phase Voltage Simulation

Mathematical Modelling, Harmonic Analysis, THD Evaluation and
Machine Learning Prediction

Tony Haddad

June 16, 2026

Contents

1	Introduction	3
2	Project Parameters	4
3	Time Axis Generation	5
4	MMC Insertion Indices	6
4.1	Concept	6
4.2	Code	6
4.3	Result	7
5	Ideal MMC Phase Voltage	8
5.1	Arm Voltages	8
5.2	Phase Voltage	8
5.3	Code	9
6	Stepped MMC Phase Voltage	10
6.1	Concept	10
6.2	Code	10
6.3	Ideal vs Stepped Voltage	11
7	Effect of the Number of Submodules	12
8	Three-Phase MMC Voltage Generation	14
8.1	Phase Shifts	14
8.2	Code	14
8.3	Result	15
9	Line-to-Line Voltages	16
9.1	Concept	16
9.2	Code	16
9.3	Result	17
10	Fast Fourier Transform Analysis	18
10.1	Concept	18
10.2	Code	18
10.3	Magnitude Scaling	18
10.4	Harmonic Spectrum Result	19

11 Total Harmonic Distortion	20
11.1 Concept	20
11.2 Code	20
12 THD vs Number of Submodules	22
12.1 Code	22
12.2 Result	23
13 Dataset Generation	24
13.1 Purpose	24
13.2 Code	24
14 Machine Learning Models	26
14.1 Goal	26
14.2 Linear Regression	26
14.3 Neural Network	27
15 Model Evaluation	28
15.1 Mean Absolute Error	28
15.2 Coefficient of Determination	28
16 Main Program Flow	29
16.1 Main Script	29
17 Conclusion	31

Chapter 1

Introduction

This project simulates the output voltage of a Modular Multilevel Converter, abbreviated as MMC. The goal is to model the phase voltage, generate stepped voltage waveforms, analyze their harmonic content, calculate the Total Harmonic Distortion, and use machine learning models to predict THD values.

An MMC is a power electronic converter used in high-voltage direct current systems. Instead of producing a voltage using only one switching device, the MMC uses many submodules connected in series. By inserting or bypassing these submodules, the converter can generate a staircase-shaped voltage that approximates a sinusoidal waveform.

The central idea of this project is:

More submodules $\Rightarrow$ smoother voltage waveform

and therefore:

More submodules $\Rightarrow$ lower harmonic distortion

Chapter 2

Project Parameters

The main simulation parameters are defined in the file `parameters.py`.

```
1 VDC = 400_000
2 F_GRID = 50
3 MODULATION_INDEX = 0.85
4 SIM_TIME = 0.1
5 DT = 0.00001
6 DEFAULT_SUBMODULES = 20
```

The DC-link voltage is:

$$V_{dc} = 400\,000 \text{ V}$$

The grid frequency is:

$$f = 50 \text{ Hz}$$

The angular frequency is:

$$\omega = 2\pi f$$

The modulation index is:

$$m = 0.85$$

The simulation time is:

$$T_{\text{sim}} = 0.1 \text{ s}$$

The simulation time step is:

$$\Delta t = 10^{-5} \text{ s}$$

Chapter 3

Time Axis Generation

The time vector is generated using NumPy.

```
1 def create_time_axis(sim_time, dt):  
2     return np.arange(0, sim_time, dt)
```

Mathematically, this creates the discrete time sequence:

$$t = 0, \Delta t, 2\Delta t, 3\Delta t, \dots, T_{\text{sim}}$$

The time step Δt defines the resolution of the simulation. A smaller time step gives more accurate results, but it also increases the number of samples.

The number of samples is approximately:

$$N_s = \frac{T_{\text{sim}}}{\Delta t}$$

For this project:

$$N_s = \frac{0.1}{0.00001} = 10\,000$$

Chapter 4

MMC Insertion Indices

4.1 Concept

The insertion index describes the fraction of submodules inserted in one arm of the converter.

For the upper arm:

$$n_{\text{upper}}(t) = \frac{1}{2} (1 - m \sin(\omega t + \phi))$$

For the lower arm:

$$n_{\text{lower}}(t) = \frac{1}{2} (1 + m \sin(\omega t + \phi))$$

where:

- m is the modulation index
- $\omega = 2\pi f$ is the angular frequency
- t is time
- ϕ is the phase shift

The upper and lower arm insertion indices are complementary:

$$n_{\text{upper}}(t) + n_{\text{lower}}(t) = 1$$

This means that when the upper arm inserts more submodules, the lower arm inserts fewer submodules, and vice versa.

4.2 Code

```
1 def calculate_insertion_indices(t, f_grid, modulation_index,
2   phase_shift=0):
3     omega = 2 * np.pi * f_grid
4     n_upper = 0.5 * (1 - modulation_index * np.sin(omega * t +
5   phase_shift))
```

```

5     n_lower = 0.5 * (1 + modulation_index * np.sin(omega * t +
6         phase_shift))
7     return n_upper, n_lower

```

4.3 Result

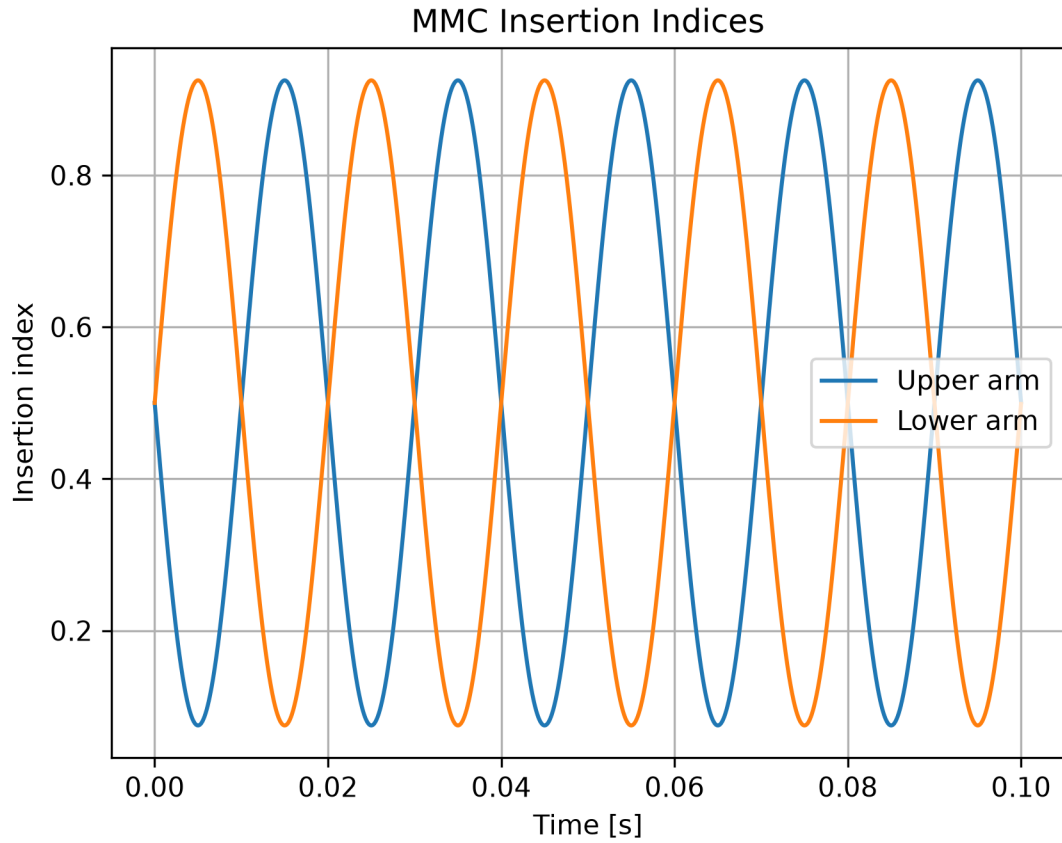


Figure 4.1: Upper and lower MMC arm insertion indices

The figure shows that the upper and lower arm insertion indices are phase-opposed. When the lower arm index increases, the upper arm index decreases.

Chapter 5

Ideal MMC Phase Voltage

5.1 Arm Voltages

The upper arm voltage is calculated as:

$$V_{\text{upper}}(t) = n_{\text{upper}}(t)V_{dc}$$

The lower arm voltage is calculated as:

$$V_{\text{lower}}(t) = n_{\text{lower}}(t)V_{dc}$$

5.2 Phase Voltage

The phase voltage is defined as:

$$V_{\text{phase}}(t) = \frac{1}{2} (V_{\text{lower}}(t) - V_{\text{upper}}(t))$$

Substituting the arm voltages gives:

$$V_{\text{phase}}(t) = \frac{1}{2} (n_{\text{lower}}(t)V_{dc} - n_{\text{upper}}(t)V_{dc})$$

$$V_{\text{phase}}(t) = \frac{V_{dc}}{2} (n_{\text{lower}}(t) - n_{\text{upper}}(t))$$

Since:

$$n_{\text{lower}}(t) - n_{\text{upper}}(t) = m \sin(\omega t)$$

the ideal phase voltage becomes:

$$V_{\text{phase}}(t) = \frac{V_{dc}}{2} m \sin(\omega t)$$

Therefore, the peak phase voltage is:

$$\hat{V}_{\text{phase}} = \frac{V_{dc}}{2} m$$

For this project:

$$\hat{V}_{\text{phase}} = \frac{400\,000}{2} \cdot 0.85$$

$$\hat{V}_{\text{phase}} = 170\,000 \text{ V}$$

5.3 Code

```

1 def calculate_ideal_phase_voltage(n_upper, n_lower, vdc):
2     v_upper = n_upper * vdc
3     v_lower = n_lower * vdc
4
5     return 0.5 * (v_lower - v_upper)

```

Chapter 6

Stepped MMC Phase Voltage

6.1 Concept

In a real MMC, the number of inserted submodules must be an integer. The insertion index is continuous, but the physical converter can only insert complete submodules.

Therefore, the number of inserted submodules is calculated as:

$$N_{\text{upper}}(t) = \text{round}(n_{\text{upper}}(t)N)$$

$$N_{\text{lower}}(t) = \text{round}(n_{\text{lower}}(t)N)$$

where N is the number of submodules per arm.

The voltage of one submodule is:

$$V_{SM} = \frac{V_{dc}}{N}$$

The stepped upper arm voltage is:

$$V_{\text{upper,step}}(t) = N_{\text{upper}}(t)V_{SM}$$

The stepped lower arm voltage is:

$$V_{\text{lower,step}}(t) = N_{\text{lower}}(t)V_{SM}$$

The stepped phase voltage is:

$$V_{\text{phase,step}}(t) = \frac{1}{2} (V_{\text{lower,step}}(t) - V_{\text{upper,step}}(t))$$

6.2 Code

```
1 def calculate_stepped_phase_voltage(n_upper, n_lower, vdc,
2   number_of_submodules):
3     n_upper_steps = np.round(n_upper * number_of_submodules)
4     n_lower_steps = np.round(n_lower * number_of_submodules)
5
6     v_sm = vdc / number_of_submodules
```

```

7     v_upper_stepped = n_upper_steps * v_sm
8     v_lower_stepped = n_lower_steps * v_sm
9
10    return 0.5 * (v_lower_stepped - v_upper_stepped)

```

6.3 Ideal vs Stepped Voltage

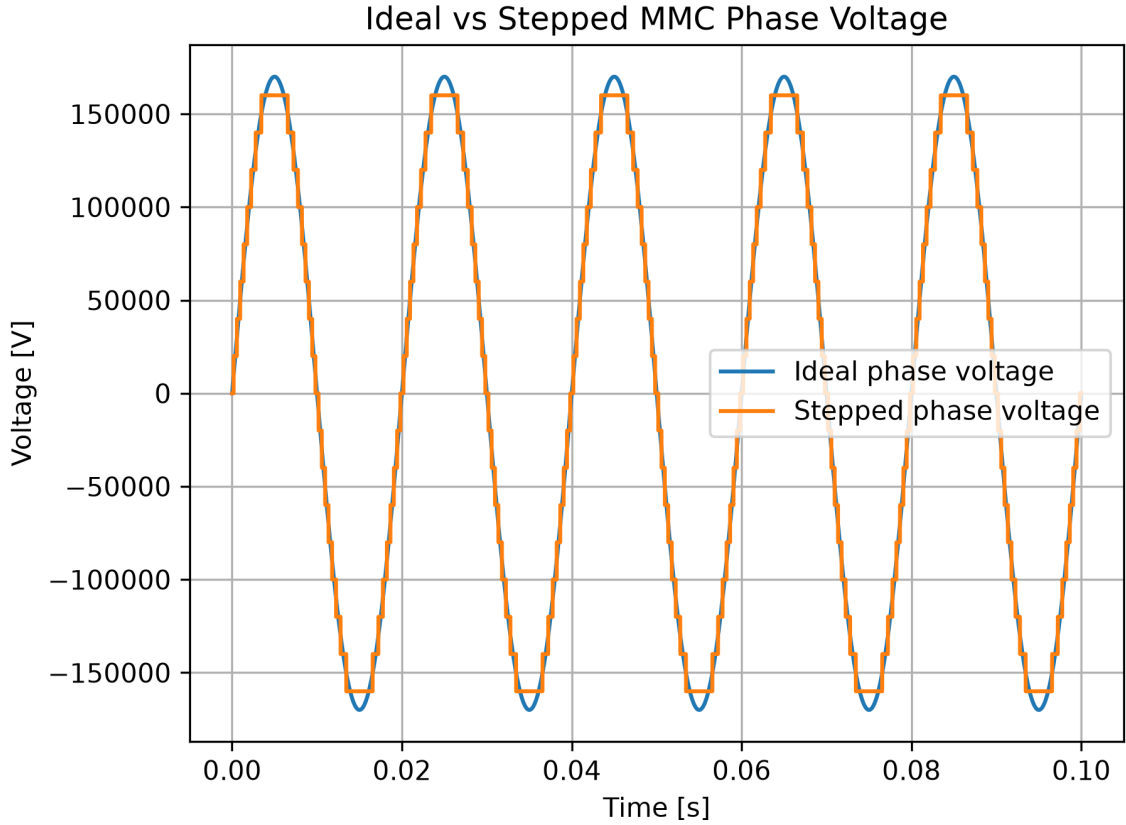


Figure 6.1: Comparison between ideal sinusoidal phase voltage and stepped MMC phase voltage

The ideal voltage is a smooth sinusoidal waveform. The stepped voltage approximates the ideal waveform using discrete voltage levels. The difference between both waveforms is caused by the finite number of submodules.

Chapter 7

Effect of the Number of Submodules

The number of submodules strongly affects the quality of the output voltage.

For a small number of submodules, the voltage waveform has large steps. For a larger number of submodules, the voltage waveform becomes smoother and closer to a sinusoidal voltage.

$$N \uparrow \Rightarrow V_{\text{phase,step}} \approx V_{\text{phase,ideal}}$$

$$N \uparrow \Rightarrow \text{voltage step size decreases}$$

The voltage step size is:

$$\Delta V = \frac{V_{dc}}{N}$$

Therefore, if N increases, ΔV decreases.

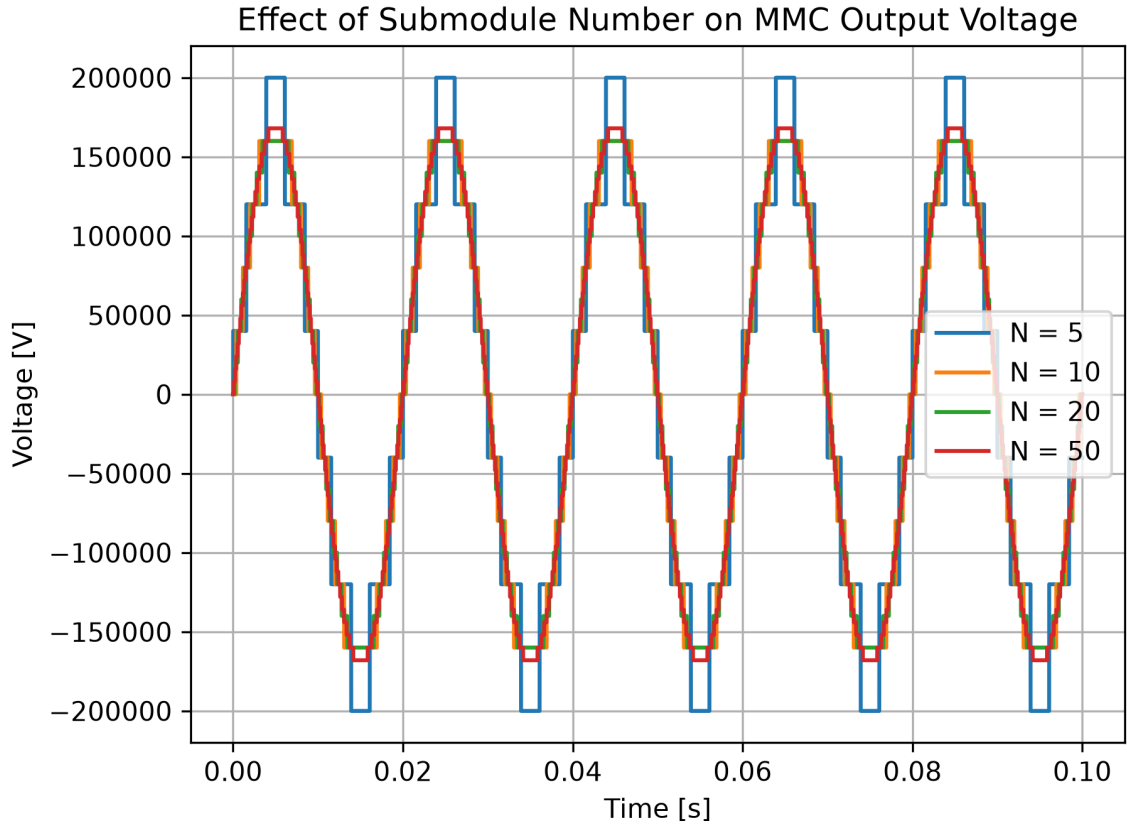


Figure 7.1: Effect of the number of submodules on the MMC output voltage

The figure shows that $N = 5$ produces a very rough voltage waveform. For $N = 50$, the voltage waveform is much smoother.

Chapter 8

Three-Phase MMC Voltage Generation

8.1 Phase Shifts

A three-phase system consists of three voltages shifted by 120° .

The phase shifts are:

$$\phi_A = 0$$

$$\phi_B = -\frac{2\pi}{3}$$

$$\phi_C = \frac{2\pi}{3}$$

The three phase voltages are:

$$V_A(t) = \frac{V_{dc}}{2} m \sin(\omega t)$$

$$V_B(t) = \frac{V_{dc}}{2} m \sin\left(\omega t - \frac{2\pi}{3}\right)$$

$$V_C(t) = \frac{V_{dc}}{2} m \sin\left(\omega t + \frac{2\pi}{3}\right)$$

8.2 Code

```
1 def calculate_three_phase_voltages(t, f_grid, modulation_index,
2   vdc, number_of_submodules):
3     phase_a = 0
4     phase_b = -2 * np.pi / 3
5     phase_c = 2 * np.pi / 3
6
7     n_upper_a, n_lower_a = calculate_insertion_indices(t, f_grid,
8       modulation_index, phase_a)
```

```

7     n_upper_b, n_lower_b = calculate_insertion_indices(t, f_grid,
8         modulation_index, phase_b)
9
10    n_upper_c, n_lower_c = calculate_insertion_indices(t, f_grid,
11        modulation_index, phase_c)
12
13    v_a = calculate_stepped_phase_voltage(n_upper_a, n_lower_a,
14        vdc, number_of_submodules)
15    v_b = calculate_stepped_phase_voltage(n_upper_b, n_lower_b,
16        vdc, number_of_submodules)
17    v_c = calculate_stepped_phase_voltage(n_upper_c, n_lower_c,
18        vdc, number_of_submodules)
19
20    return v_a, v_b, v_c

```

8.3 Result

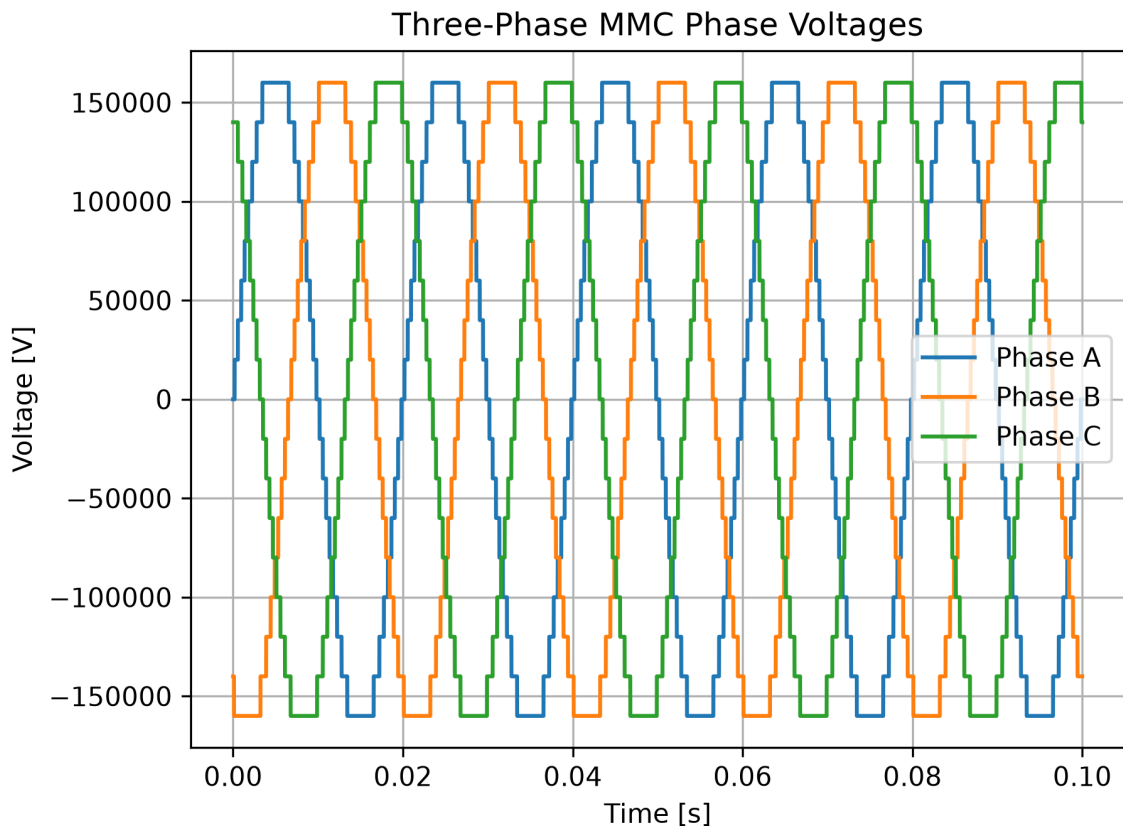


Figure 8.1: Three-phase MMC phase voltages

The three voltages have the same amplitude and frequency, but they are shifted by 120° .

Chapter 9

Line-to-Line Voltages

9.1 Concept

Line-to-line voltages are calculated from the phase voltages.

$$V_{ab}(t) = V_a(t) - V_b(t)$$

$$V_{bc}(t) = V_b(t) - V_c(t)$$

$$V_{ca}(t) = V_c(t) - V_a(t)$$

The line-to-line voltage is important because many AC systems and loads are connected between two phases, not between one phase and neutral.

For ideal sinusoidal voltages, the line-to-line voltage amplitude is:

$$\hat{V}_{LL} = \sqrt{3}\hat{V}_{phase}$$

Since:

$$\hat{V}_{phase} = 170\,000\text{ V}$$

then:

$$\hat{V}_{LL} = \sqrt{3} \cdot 170\,000$$

$$\hat{V}_{LL} \approx 294\,449\text{ V}$$

This matches the plotted line-to-line voltage, which reaches approximately 300 000 V because of the stepped approximation.

9.2 Code

```
1 def calculate_line_voltages(v_a, v_b, v_c):  
2     v_ab = v_a - v_b  
3     v_bc = v_b - v_c  
4     v_ca = v_c - v_a  
5
```

```
return v_ab, v_bc, v_ca
```

9.3 Result

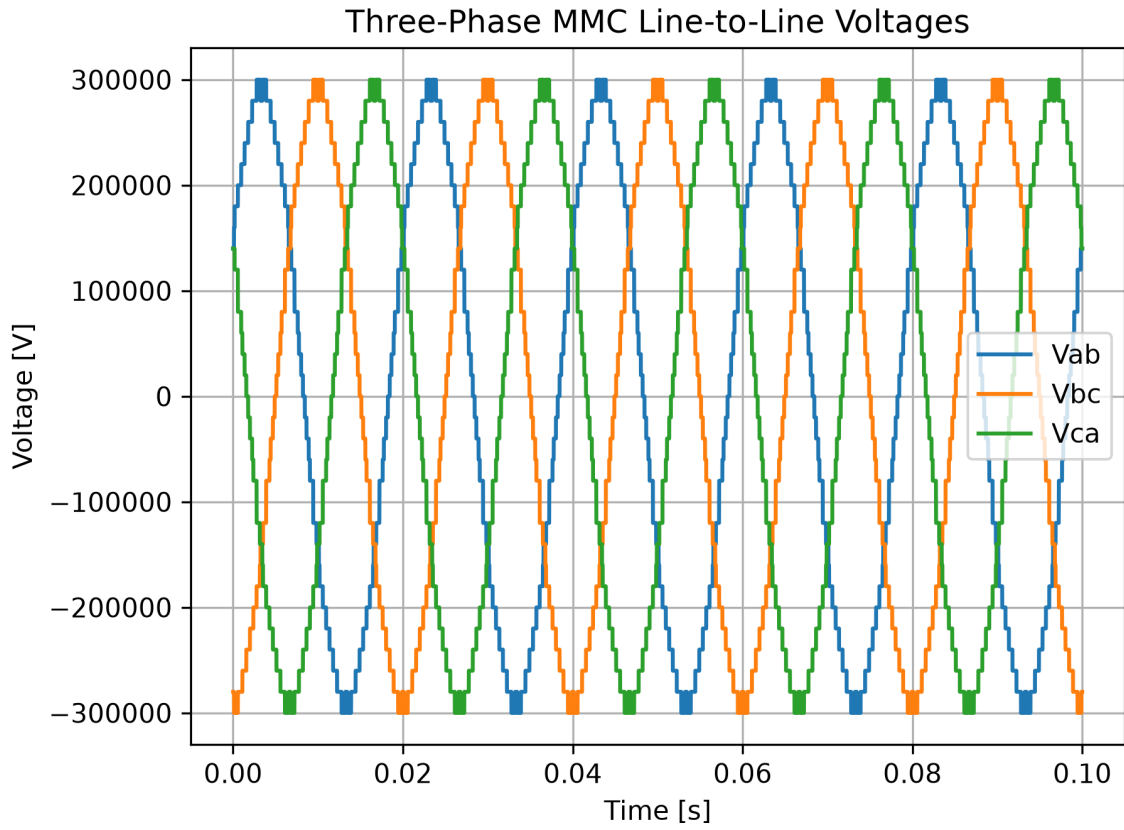


Figure 9.1: Three-phase MMC line-to-line voltages

Chapter 10

Fast Fourier Transform Analysis

10.1 Concept

The Fast Fourier Transform, abbreviated as FFT, is used to convert the voltage signal from the time domain into the frequency domain.

The time-domain voltage is:

$$v(t)$$

The frequency-domain representation is:

$$V(f)$$

The FFT shows which frequency components exist inside the voltage waveform.

For an ideal sinusoidal voltage at 50 Hz, only the fundamental frequency should exist. For a stepped voltage, additional harmonics appear because the waveform is not perfectly sinusoidal.

10.2 Code

```
1 def calculate_fft(signal, dt):
2     number_of_samples = len(signal)
3
4     frequencies = np.fft.rfftfreq(number_of_samples, dt)
5     fft_values = np.fft.rfft(signal)
6
7     magnitudes = np.abs(fft_values) / number_of_samples
8     magnitudes[1:] = 2 * magnitudes[1:]
9
10    return frequencies, magnitudes
```

10.3 Magnitude Scaling

The magnitude is calculated as:

$$|V_k| = \frac{|FFT(v)|}{N_s}$$

For all frequencies except DC, the magnitude is doubled:

$$|V_k| = 2|V_k|$$

This is done because the real FFT returns only the positive-frequency part of the spectrum.

10.4 Harmonic Spectrum Result

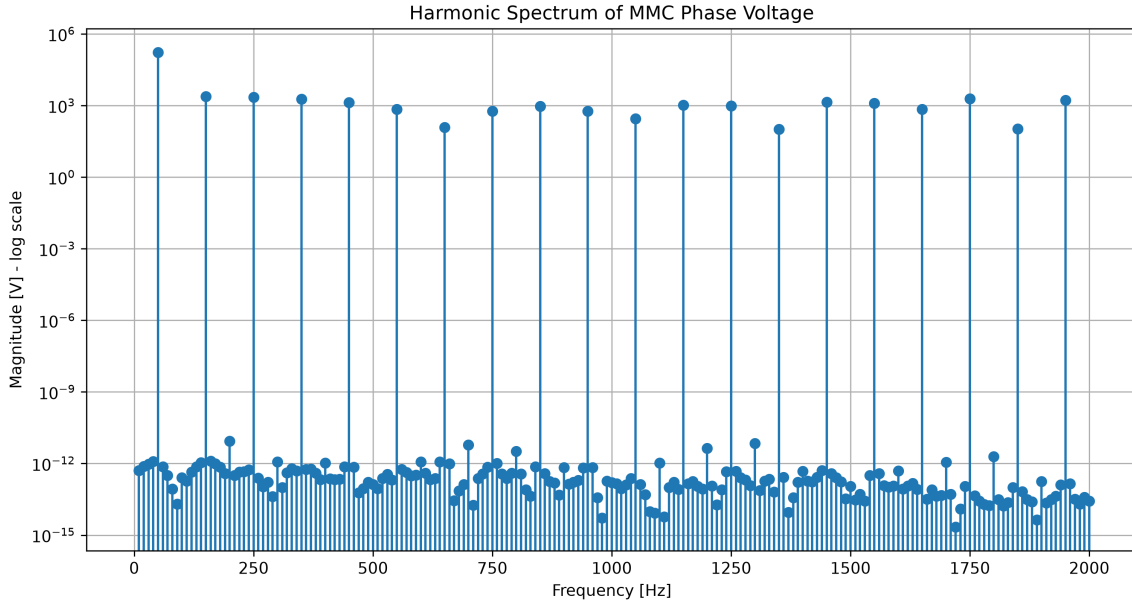


Figure 10.1: Harmonic spectrum of the MMC phase voltage

The dominant component is the fundamental frequency at 50 Hz. Additional peaks appear at higher frequencies because the stepped waveform contains harmonics.

Chapter 11

Total Harmonic Distortion

11.1 Concept

Total Harmonic Distortion, abbreviated as THD, measures how distorted a waveform is compared to a pure sinusoidal waveform.

A pure sinusoidal voltage has:

$$THD = 0$$

A distorted waveform has:

$$THD > 0$$

The THD equation is:

$$THD = \frac{\sqrt{V_2^2 + V_3^2 + V_4^2 + \dots + V_n^2}}{V_1}$$

where:

- V_1 is the fundamental voltage magnitude
- $V_2, V_3, \dots, V_n$ are the harmonic voltage magnitudes

In percent form:

$$THD_{\%} = THD \cdot 100$$

11.2 Code

```
1 def calculate_thd(signal, dt, fundamental_frequency=50,
2   max_harmonic_order=40):
3     frequencies, magnitudes = calculate_fft(signal, dt)
4     fundamental_index = np.argmin(np.abs(frequencies -
5     fundamental_frequency))
6     fundamental_magnitude = magnitudes[fundamental_index]
```

```

7     harmonic_magnitudes = []
8
9     for harmonic_order in range(2, max_harmonic_order + 1):
10         harmonic_frequency = harmonic_order *
            fundamental_frequency
11         harmonic_index = np.argmin(np.abs(frequencies -
            harmonic_frequency))
12         harmonic_magnitudes.append(magnitudes[harmonic_index])
13
14     harmonic_rms_sum = np.sqrt(np.sum(np.array(
            harmonic_magnitudes) ** 2))
15
16     return harmonic_rms_sum / fundamental_magnitude

```

Chapter 12

THD vs Number of Submodules

The project calculates THD for different numbers of submodules.

$$N = 5, 10, 20, 50, 100$$

The physical meaning is simple:

$$N \uparrow \Rightarrow \Delta V \downarrow \Rightarrow \text{harmonics decrease} \Rightarrow THD \downarrow$$

12.1 Code

```
1 def calculate_thd_for_multiple_submodules(  
2     n_upper,  
3     n_lower,  
4     vdc,  
5     dt,  
6     n_values,  
7     fundamental_frequency=50  
8 ):  
9     thd_results = {}  
10  
11     for n in n_values:  
12         v_phase_stepped = calculate_stepped_phase_voltage(n_upper  
13             , n_lower, vdc, n)  
14         thd = calculate_thd(v_phase_stepped, dt,  
15             fundamental_frequency)  
16         thd_results[n] = thd  
  
17     return thd_results
```

12.2 Result

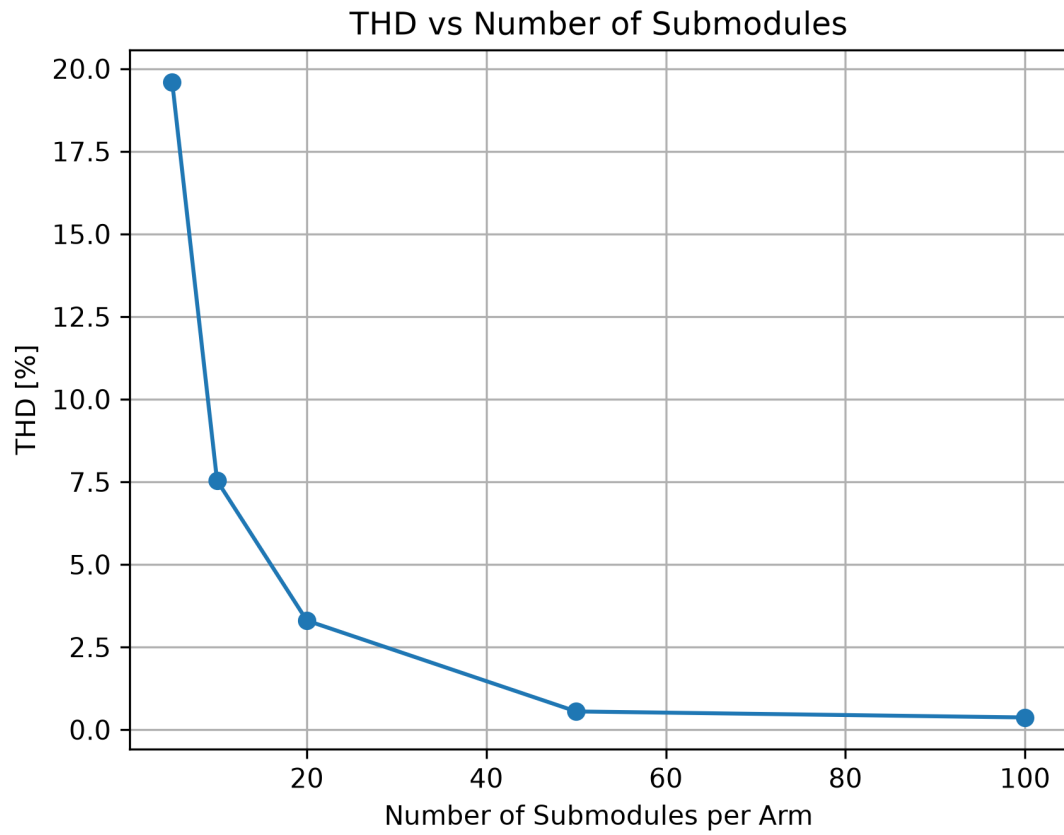


Figure 12.1: THD as a function of the number of submodules per arm

The figure shows that the THD decreases strongly when the number of submodules increases. The largest improvement occurs when moving from a very small number of submodules to a moderate number of submodules.

Chapter 13

Dataset Generation

13.1 Purpose

The project also generates a dataset for machine learning. The dataset contains different combinations of:

$$N$$

and

$$m$$

where:

- N is the number of submodules
- m is the modulation index

For every pair (N, m) , the THD is calculated and stored.
The dataset therefore has the form:

$$(N, m) \rightarrow THD$$

13.2 Code

```
1 def generate_thd_dataset(  
2     n_values,  
3     modulation_values,  
4     output_path="results/dataset.csv"  
5 ):  
6     os.makedirs(os.path.dirname(output_path), exist_ok=True)  
7  
8     t = create_time_axis(SIM_TIME, DT)  
9  
10    rows = []  
11  
12    for n in n_values:  
13        for m in modulation_values:
```

```

14         n_upper, n_lower = calculate_insertion_indices(t,
15             F_GRID, m)
16
17         v_phase_stepped = calculate_stepped_phase_voltage(
18             n_upper,
19             n_lower,
20             VDC,
21             n
22         )
23
24         thd = calculate_thd(
25             v_phase_stepped,
26             DT,
27             fundamental_frequency=F_GRID
28         )
29
30         rows.append({
31             "N": n,
32             "modulation_index": m,
33             "THD": thd,
34             "THD_percent": thd * 100
35         })
36
37     with open(output_path, mode="w", newline="") as file:
38         writer = csv.DictWriter(
39             file,
40             fieldnames=["N", "modulation_index", "THD", "THD_percent"]
41         )
42
43         writer.writeheader()
44         writer.writerows(rows)
45
46     return rows

```

Chapter 14

Machine Learning Models

14.1 Goal

The machine learning part predicts THD from:

$$X = [N \quad m]$$

The output is:

$$y = THD_{\%}$$

So the model learns the mapping:

$$f(N, m) = THD_{\%}$$

14.2 Linear Regression

Linear regression assumes a relationship of the form:

$$THD_{\%} = aN + bm + c$$

where:

- a is the coefficient for the number of submodules
- b is the coefficient for the modulation index
- c is the intercept

```
1 def train_linear_regression(dataset_path):
2     X, y = load_dataset(dataset_path)
3
4     model = LinearRegression()
5     model.fit(X, y)
6
7     predictions = model.predict(X)
8
9     mae = mean_absolute_error(y, predictions)
10    r2 = r2_score(y, predictions)
```

```
11
12     return model, mae, r2
```

14.3 Neural Network

The neural network is used because the relationship between N , m , and THD is not perfectly linear.

The model structure is:

$$2 \rightarrow 32 \rightarrow 16 \rightarrow 1$$

This means:

- 2 input features: N and m
- first hidden layer with 32 neurons
- second hidden layer with 16 neurons
- one output value: predicted THD

```
1 def train_neural_network(dataset_path):
2     X, y = load_dataset(dataset_path)
3
4     X_train, X_test, y_train, y_test = train_test_split(
5         X,
6         y,
7         test_size=0.25,
8         random_state=42
9     )
10
11     model = Pipeline([
12         ("scaler", StandardScaler()),
13         ("neural_network", MLPRegressor(
14             hidden_layer_sizes=(32, 16),
15             activation="relu",
16             max_iter=5000,
17             random_state=42
18         ))
19     ])
20
21     model.fit(X_train, y_train)
22
23     predictions = model.predict(X_test)
24
25     mae = mean_absolute_error(y_test, predictions)
26     r2 = r2_score(y_test, predictions)
27
28     return model, mae, r2
```

Chapter 15

Model Evaluation

15.1 Mean Absolute Error

The Mean Absolute Error, abbreviated as MAE, is:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where:

- y_i is the real THD value
- $\hat{y}_i$ is the predicted THD value

A smaller MAE means better predictions.

15.2 Coefficient of Determination

The R^2 score measures how well the model explains the data.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where:

$$\bar{y}$$

is the average value of the real THD values.

If:

$$R^2 = 1$$

then the model predicts perfectly.

If:

$$R^2 = 0$$

then the model is not better than predicting the average value.

Chapter 16

Main Program Flow

The main script performs the following steps:

1. Create the time axis
2. Calculate upper and lower arm insertion indices
3. Calculate ideal MMC phase voltage
4. Calculate stepped MMC phase voltage
5. Perform FFT analysis
6. Calculate THD
7. Plot all voltage and harmonic results
8. Generate a THD dataset
9. Train linear regression and neural network models
10. Predict THD for a new input

16.1 Main Script

```
1 t = create_time_axis(SIM_TIME, DT)
2
3 n_upper, n_lower = calculate_insertion_indices(
4     t,
5     F_GRID,
6     MODULATION_INDEX
7 )
8
9 v_phase = calculate_ideal_phase_voltage(
10     n_upper,
11     n_lower,
12     VDC
13 )
14
15 v_phase_stepped = calculate_stepped_phase_voltage(
```

```

16     n_upper,
17     n_lower,
18     VDC,
19     DEFAULT_SUBMODULES
20 )
21
22 frequencies, magnitudes = calculate_fft(
23     v_phase_stepped,
24     DT
25 )
26
27 thd = calculate_thd(
28     v_phase_stepped,
29     DT,
30     fundamental_frequency=F_GRID
31 )
32
33 print(f"THD for N={DEFAULT_SUBMODULES}: {thd*100:.2f}%")

```

Chapter 17

Conclusion

This project demonstrates how an MMC can generate an AC voltage from a DC voltage using insertion indices and submodule switching.

The most important equations are:

$$n_{\text{upper}}(t) = \frac{1}{2} (1 - m \sin(\omega t))$$

$$n_{\text{lower}}(t) = \frac{1}{2} (1 + m \sin(\omega t))$$

$$V_{\text{phase}}(t) = \frac{1}{2} (V_{\text{lower}}(t) - V_{\text{upper}}(t))$$

$$THD = \frac{\sqrt{V_2^2 + V_3^2 + \dots + V_n^2}}{V_1}$$

The simulation results show that increasing the number of submodules improves the quality of the output voltage and reduces THD.

This makes the project useful for understanding:

- MMC voltage generation
- insertion index modulation
- stepped voltage approximation
- harmonic analysis
- THD calculation
- simple machine learning prediction for power electronics